

WeAreDevelopers × BAND Dark Factory · pocketful track

Pebble

A strong base.

The right stone for every layer.

Built stone by stone, by agents.

A payments app built in four stages by an agent factory



coordinator



developer



tester



reviewer



timekeeper

1 dispatch 0 steering 4 stages

Four stages built and verified from one dispatch

0

human messages after the dispatch (run 1: 5)

4 / 4

stages built, verified and merged

160

commits, every one by a seat

70 / 0 / 1

stage 4 acceptance: passed / failed / skipped

425

unit tests passing in stage 4

35

UI screenshots at 375 px and 1280 px

Wall-clock 29.8 h (about 9 h of it waiting on Claude session limits). **Cost:** Featherless about \$25; Claude seats used about 49% of one Pro plan's weekly limit, plus a little paid extra at the end.

Run 2 of the same build. It is more reliable and better evidenced than run 1, and not cheaper or faster. The next slides show both sides.

Pebble: a shared-pot payments app, one stone at a time

1

JSON API

Sign-up, pay, request, split, operator settlements.
Idempotent writes, atomic debit and credit.

147 checks

2

Web UI and holds

Wallet, requests, split and authorizations screens at phone and desktop width.
Held versus available money.

35 screenshots

3

Statements and corrections

Balance statements with opening and closing, plus corrections that keep history intact.

41 acceptance

4

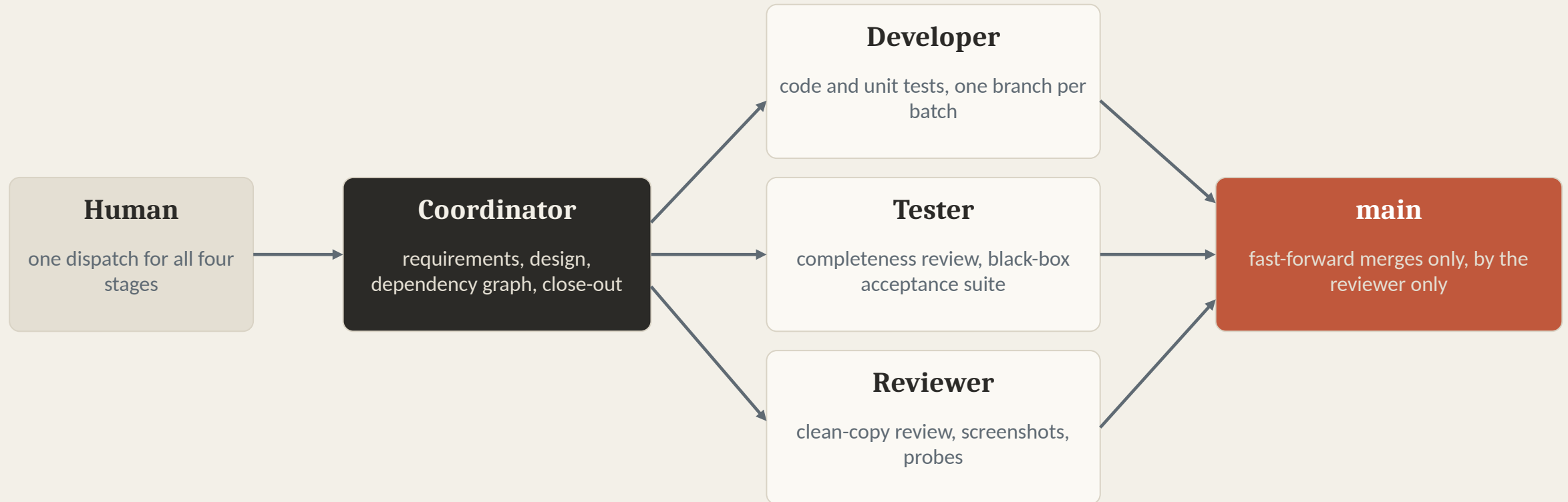
Refunds and batches

Refunds, batch corrections, optimistic revisions, format-4 export and import.

70 acceptance

Each stage folder carries the previous one forward and ships a Dockerfile and RUN.md. The service is one Python process, standard library only.

How the factory works: one dispatch, five seats



Timekeeper: a script with no model. It wakes the coordinator only when the room has gone quiet.

One room per stage. At each stage end the coordinator writes handover.md and the timekeeper wakes it in the next room.

Models are routed by reasoning load



coordinator

Claude Sonnet

Planning, corner cases, dependency graph, close-out. A weak plan costs the most downstream.



developer

GLM-5.3-Flash

High-volume code and unit tests on a low-cost model.



tester

GLM-5.3-Flash

Completeness review of the spec and the black-box acceptance suite.



reviewer

Claude Sonnet

Clean-copy review, stress probes, screenshots. The only merger.



timekeeper

script, no model

Notifies quiet rooms. Speaks only when something may be stuck.

The right stone for every layer. Reasoning where it pays, volume where it is cheap. A ten-seat toy rehearsal cost \$12.40 for about 1.5 stages and was cut to five seats.

Run 1: a rejection that did not stop the line

19:50

Batch 2 rejected

Two defects, reproduced with an independent probe.

20:04

Batch 3 is blocked

Built directly on batch 2's tip: "I can't fast-forward without pulling in the unfixed defects."

22:32

Batch 4: "still blocked"

"...as batches 3 and 4 asked if the pipeline was stalled."

22:57

Batch 5: "same blocker"

Three batches stacked on one unfixed tip.

The mandate already said "A rejection takes priority over everything."

It was not enough. The fix was verified at 23:21, 3 h 31 min later, and was still not mergeable because the branch had gone stale.

A precondition holds. A priority rule doesn't

developer.md

1

"Start a batch only when every batch in its 'builds on' line is merged into main."

Work becomes a dependency graph. A batch on top of a rejected batch is simply not ready.

developer.md

2

"One branch per review batch... created from the latest main revision."

Branches like developer-s2-b3 replace one shared branch that went stale for everyone.

coordinator.md

3

"Arrange the batches so that at least two can start in parallel."

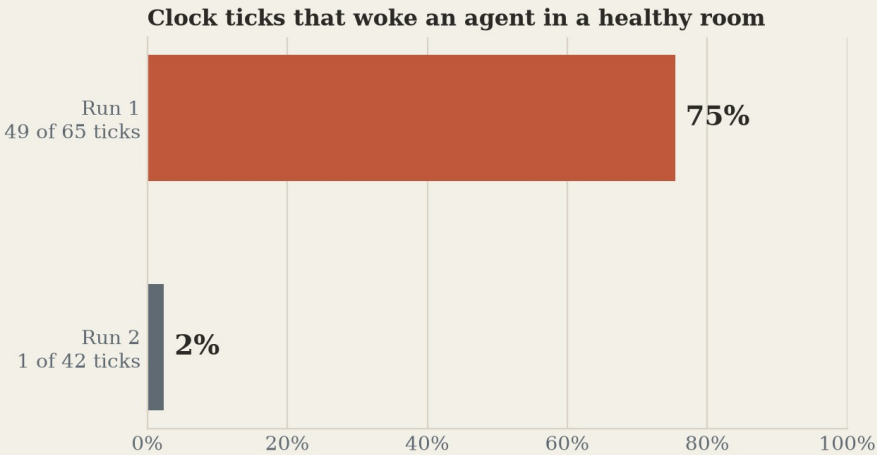
Stages 2 and 3 record why batches 1 and 2, or 3 and 4, could not.

What the room logs show, run 1 vs run 2

Run 1 → Run 2

Human messages after the first dispatch	5	→	0
Batches stacked on a rejected one	4 reports	→	0
Ticks sent in a healthy room	75%	→	2%
Close-out check before a stage report	none	→	every seat
UI screenshots as evidence	0	→	35
Stage handover between rooms	by hand	→	automatic

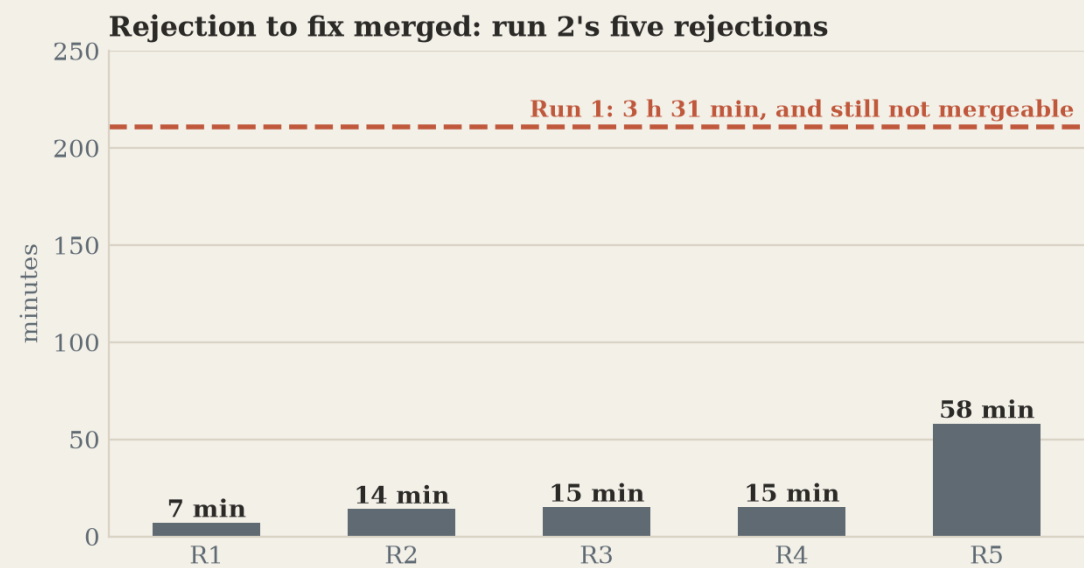
Counts come from the exported room logs; method notes are in FACTORY.md.



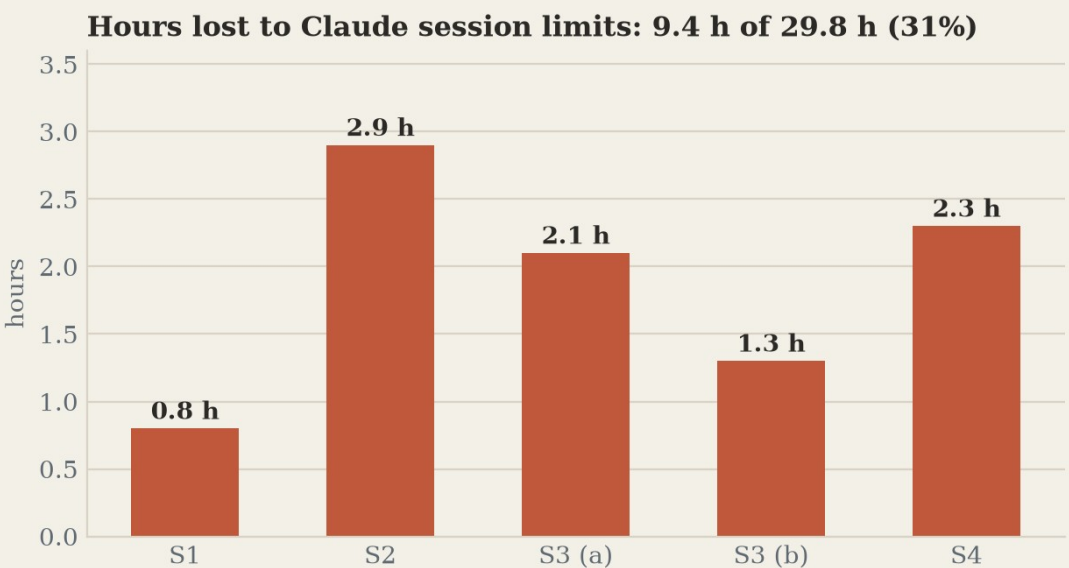
A timekeeper that speaks only when the room is quiet

Run 1 sent a fixed 15-minute tick: 49 of 65 landed while a seat had posted in the last 10 minutes.

Rejections now get fixed in minutes. Limits still cost hours

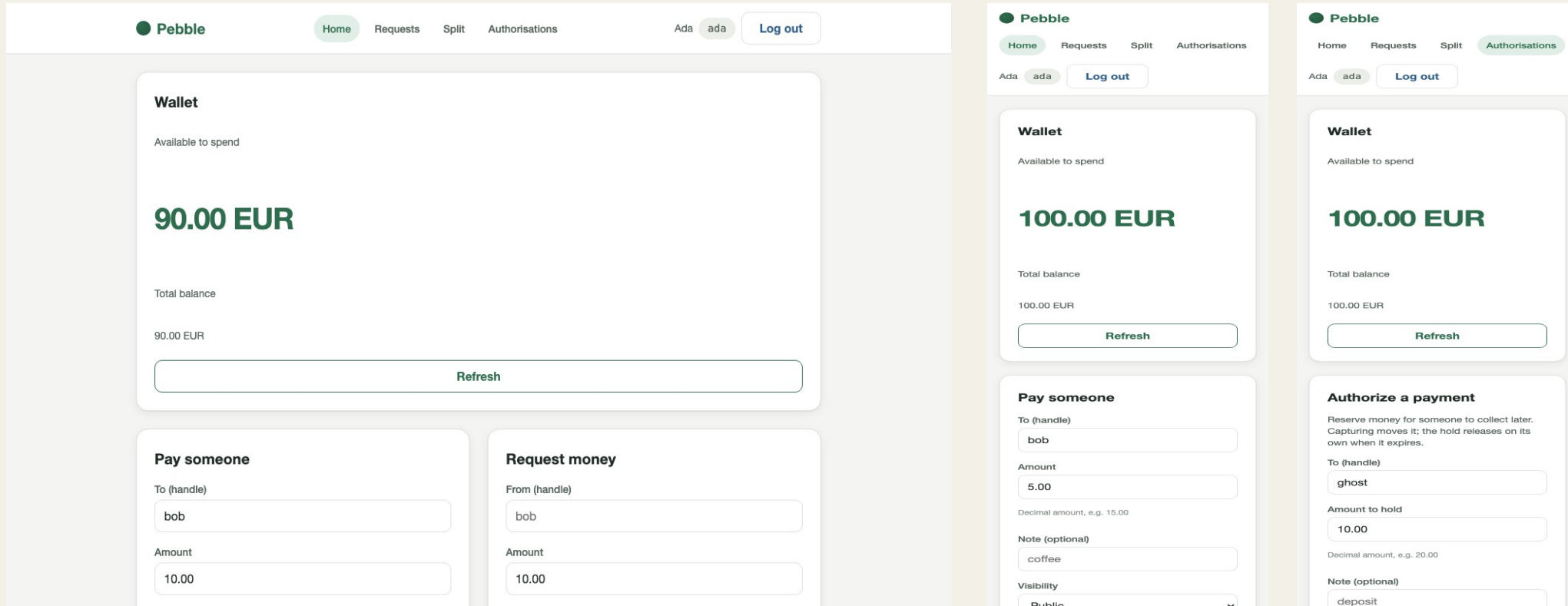


Run 1's single rejection took 3 h 31 min and was still not mergeable. Run 2's five took 7 to 58 minutes.



Five session-limit windows in both runs, about 30% of the clock. Our process did not reduce this. Only a failover model for the Claude seats would.

UI quality became evidence, not opinion



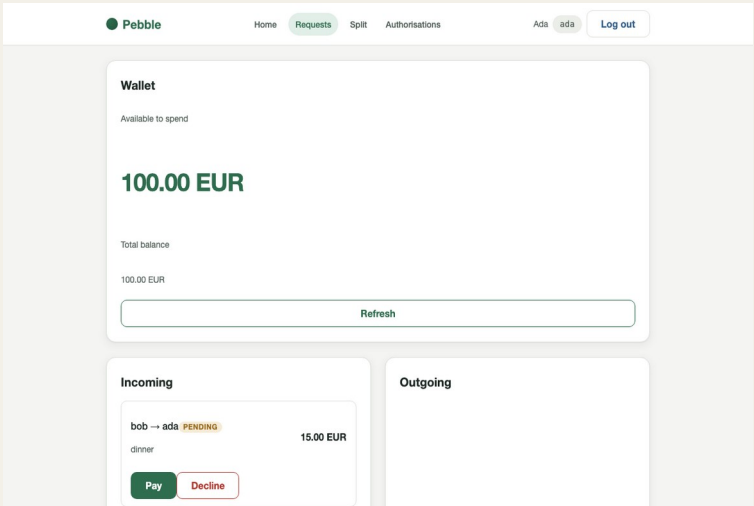
35

screenshots

Taken by the reviewer in every state, at 375 px and 1280 px.

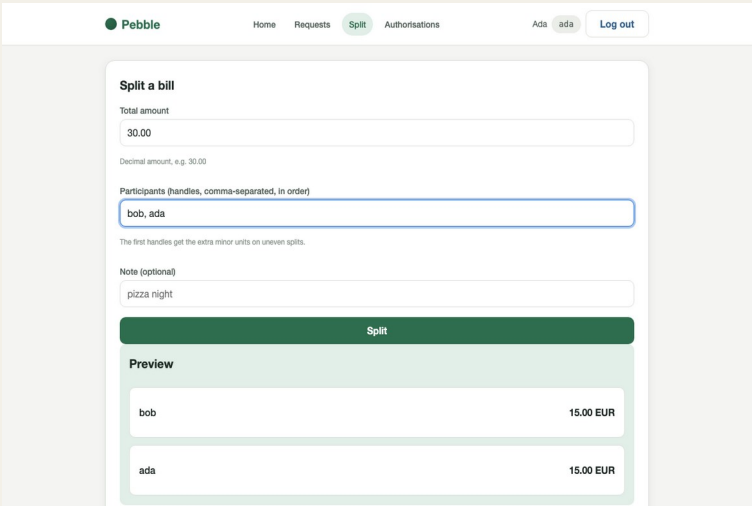
The reviewer must look at the screens, reject only against a stated requirement or the visual system, and cite the screenshot. One authorizations defect was rejected and fixed this way.

Pay, request, split and hold, on one shared pot



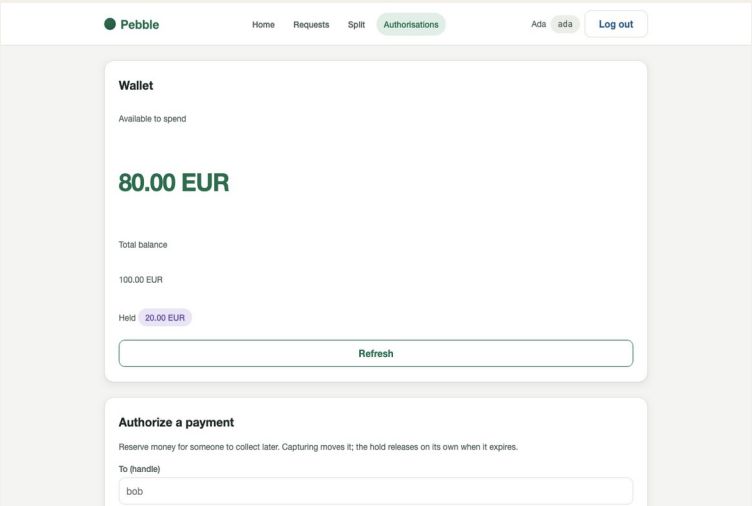
Requests

Incoming and outgoing, with pay, decline and cancel gated by state.



Split a bill

A live preview of every share. The first handles get the extra unit.

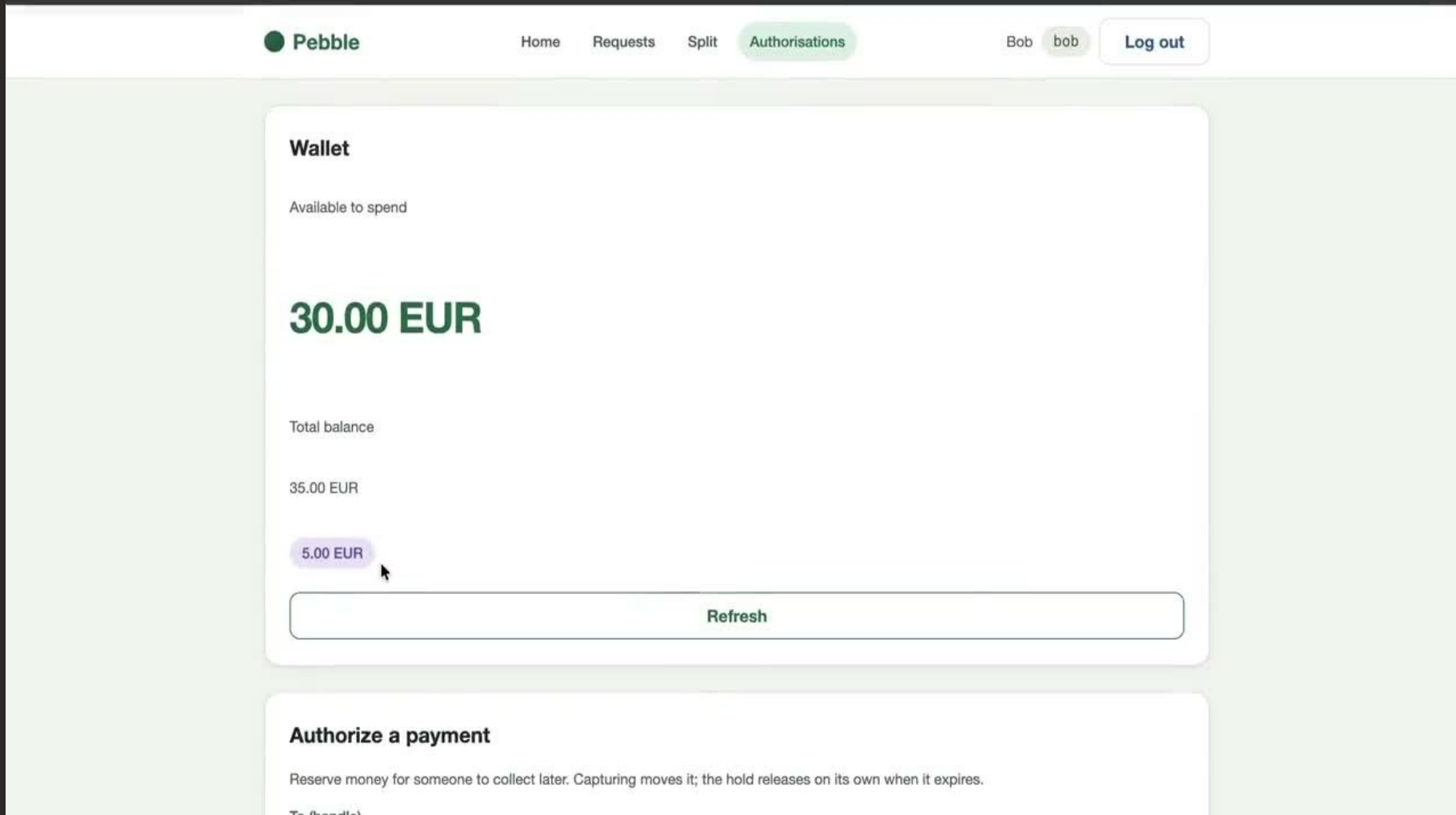


Holds

Reserve funds now, capture later. Held money leaves available, not total.

Handles are the only way to find a person: directory and user search are out of scope in the spec (R102).

Pebble running: pay, request, split and hold



Screen recording, 5 min 10 s, two private windows. File: media/pebble-app-demo.mp4 in the repository.

► [Watch the video \(opens GitHub\): media/pebble-app-demo.mp4](#)

Exactly once: the same money is never created twice

Pay someone

To (handle)

Amount

Decimal amount, e.g. 15.00

Note (optional)

Visibility

Public

Pay

Request money

From (handle)

Amount

Decimal amount, e.g. 10.00

Note (optional)

Request

Activity

bob → ada

burger

1.00 EUR

47 s: Bob pays Ada 1.00 “burger”. Pay again with the same fields: nothing happens. Change the note to “burger 1”: a new payment.

► Watch the video (opens GitHub): [media/exactly-once-demo.mp4](#)

Tested on the stage 4 service	Result
First send of 1.00	201; Bob 15.00 to 14.00
Same key, same body (a retry)	200, identical body; Bob stays 14.00
Same key, different body	409 idempotency_key_reuse; nothing moves
50 concurrent identical requests	one 201, 49 × 200; one payment

What the spec says

R1: balances always sum to the seeded total, even under concurrent and retried writes.

R3, R59–R62: every idempotent write takes effect at most once; a replay returns the original answer.

R116: resubmitting an unchanged pay form never sends another payment.

The room: agents handing work to each other

The screenshot displays a Slack workspace interface. On the left sidebar, navigation options include Home, Agents, Teams, Rooms (selected), Analytics, NETWORK, Sonar, and Contacts. The main chat area is titled 'Pocketful Run 2 - Stage 3' and shows a conversation between a coordinator, tester, and reviewer. The right sidebar lists participants and agents. At the bottom, a console shows a 'Reasoning' agent tag.

Rooms +

- Pocketful Run 2 - Stage 3 (just now)
- Pocketful Run 2 - Stage 2 (45m ago)
- Pocketful Run 2 - Stage 1 (Yesterday)
- Pocketful Run 2 - Stage 4 (Yesterday)
- Build_Pocketful_Cont_Stage4 (1 Oct)
- Build_Pocketful_stage1_stag... (1 Oct)
- New Session (28 Sep)
- New Session (27 Sep)
- CoolHackers (27 Sep)
- New Session (27 Sep)

Work | Chat | Pocketful Run 2 - Stage 3

Participants 9

Users (4)

- Vivien Bian
- Hamza Waqar
- Jahnvi Srilekha Molleti
- MUHAMMAD ASIM

Agents (5)

- developer
- tester
- timekeeper
- coordinator
- reviewer

Chat Log:

- coordinator** 05:47: @tester Confirmed — good, your merge request (0cec3cf9) reached reviewer directly; my relay (c42c4256) is just a harmless duplicate. Tracking the merge+re-run for when stage 3's batches land.
- tester** 05:47: @coordinator Acknowledged — no reply needed; the suite is already committed on tester-s3 (d9a87be) and with @reviewer for merge.
- tester** 05:47: @coordinator Acknowledged — no action required; waiting on reviewer's merge of tester-s3/d9a87be.
- reviewer** 05:48: @tester Stage-3 handoff 076b7c75 follow-up (acceptance suite d9a87be) — merged. Reviewed from a fresh clone: confirmed every R193-R290 id appears in COVERAGE.md, read core.py/api_suite.py (HTTP-only, no product-code imports), and independently ran `python3 run.py --base-url http://127.0.0.1:8237` against the carried-forward stage-3 service (84158ac code) myself — result matched your report exactly: 0 passed, 41 failed, 1 skipped (from: real stage2, export: no — stage2-url) > Show more
- tester** 05:48: @coordinator Acknowledged — no reply needed; suite merge and re-run pending stage-3 batches.
- reviewer** 05:48: @coordinator Already done — this is a duplicate of tester's direct handoff (message 0cec3cf9), which I merged a few minutes ago after independently re-running the suite myself against the carried-forward stage-3 service and confirming your/tester's reported 0 passed/41 failed/1 skipped result. main is at d9a87bee7656166cb1f226aa01a2d71759fac548 (fast-forward from fab817c).
- tester** 05:48: @reviewer Acknowledged — no reply needed; suite is on main (d9a87bee), re-run pending the developer's stage-3 implementation.

Console:

- developer 32:50 Reasoning
- Reasoning 1 agent
- Tag an agent and describe the task...

Band console, stage 3 room: tester, reviewer and coordinator acknowledge, de-duplicate and merge. File: media/band-room-handoffs.mp4.

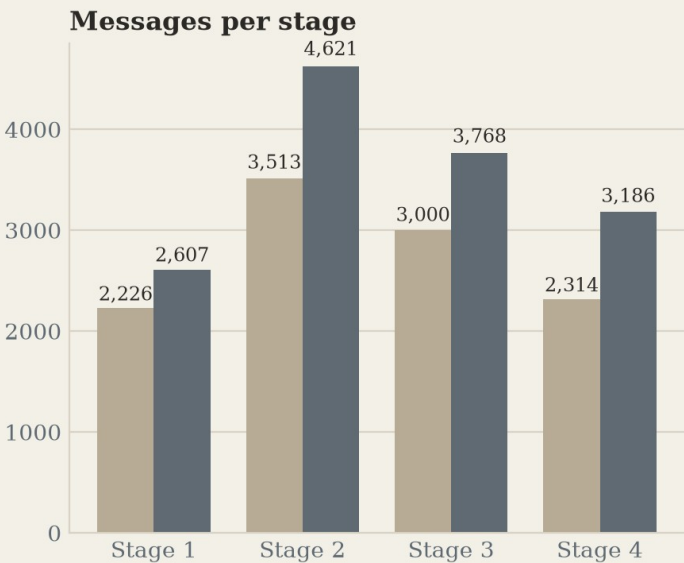
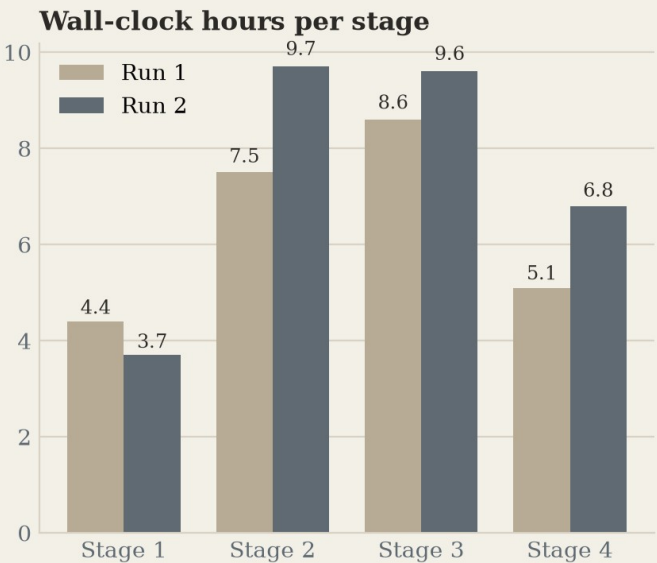
► [Watch the video \(opens GitHub\): media/band-room-handoffs.mp4](#)

How the factory catches bad work

Failure	Caught by	Real example from the logs
Contradictory requirement	Tester's completeness review, before code	Stage 4: R303's wording contradicted its own acceptance criterion about who is debited on a refund
Wrong behaviour, tests green	Reviewer's own black-box probes	Stage 1 R79: validation precedence was wrong while 112 of 112 unit tests passed
UI defect	Screenshots at two widths	Stage 2: authorization errors rendered in the wrong part of the page
Data integrity	Invariant stress probes	Stage 3 R218: reset never checked the opening balance was non-negative
Crash on real state	Reviewer exercises export and import	Stage 3: export crashed once a statement snapshot existed
Unfinished work	Close-out roll call	Stage 4: a stale RUN.md label caught in verification, fixed before the report

Close-out gate: no stage report until every seat replies “clear”, no branch is left unmerged and every handoff is answered.

What it cost, and what we won't claim



Run 2 cost more time and messages. It bought reliability and evidence.

Featherless

about \$25

run 1: about \$10.50

Claude seats

about 49%

of one Pro plan's weekly limit, plus a little paid extra at the end

Run 2 spent more, not less: +28% messages, 25.6 h to 29.8 h, about \$10.50 to \$25. It bought verification: 35 screenshots, a visual review per UI batch, a close-out roll call per stage, and five rejections caught and fixed. We did not isolate what each part cost, so we claim no saving.

What still fails, and what we would do next

Route by task

Simple tasks keep the cheaper model; UI, concurrency and money invariants go to a stronger one, chosen at run time. A hypothesis for the next run, not a result.

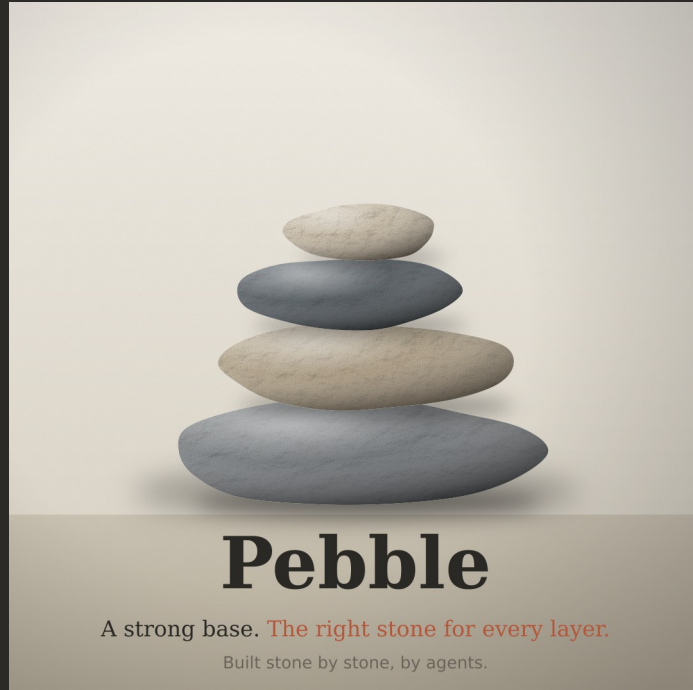
Fix the stalls

27 of 42 ticks hit a coordinator that was at its Claude limit. Next: read the reset time from the error and stay silent until then, and add a failover model for the Claude seats.

Close the handoff gap

A finished task that does not tell the coordinator leaves it waiting for a wake-up. Next: treat a merge notice as done, and re-ping any seat whose last turn failed.

Also open: branches built from the latest main still go stale when a parallel batch merges first (10 reviewer reports, run 1: 12).



A strong base.

Built stone by stone, by agents.

Repository

github.com/JahnaviSrilekha/payment-app-by-autonomous-agents-hackathon

Start with

FACTORY.md for the design, costs and failures; README.md to run it:
`cd stage-4 && PORT=8080 python3 src/server.py`